

Anonymized API Security Review

Portfolio Case Study

Year: 2026

Review type: Passive API security review / responsible disclosure

Target: An anonymized multi-tenant SaaS platform exposing public REST and JSON-RPC APIs

Method: Review of publicly available pages, API documentation, schemas, and previously observed API descriptions. No exploitation of production systems was performed.

Anonymization note

The company name, domains, unique endpoint names, project-specific identifiers, and other details that could reasonably identify the organization have been removed or generalized. The technical behavior, security reasoning, validation requirements, and remediation guidance have been preserved.

1. Executive Summary

I performed a passive security review of a public API surface for a multi-tenant SaaS platform. The goal was to identify security-relevant design issues and authorization questions that could be assessed from public documentation without interacting with real customer data or attempting to compromise the environment.

The review identified several areas that warranted internal validation:

1. object-level authorization for project and analytics resources;
2. state-changing operations exposed through HTTP GET;
3. operational fields exposed by external API responses;
4. legacy error handling that returns HTTP 200 for some application errors;
5. token handling and debug data in JSON-RPC flows;
6. privacy impact from personally identifiable information (PII) in API schemas.

The review deliberately distinguishes between facts visible in public documentation and hypotheses that require controlled runtime testing. No compromise, unauthorized data access, or exploitation of production systems was claimed.

2. Ethical and Testing Boundaries

I did not have explicit authorization to conduct a full active penetration test against the target infrastructure. Because of that, I intentionally did not attempt to:

- bypass authentication or authorization controls;
- enumerate or test object identifiers belonging to real customers;
- use third-party or discovered token-like values;
- exploit potential BOLA/IDOR, CSRF, or other vulnerabilities;
- modify, delete, or export production customer data;
- gain unauthorized access to administrative interfaces;
- execute code on servers, obtain shell access, establish persistence, or otherwise implant into the infrastructure;
- scan internal networks, perform lateral movement, or access adjacent services;
- perform brute-force, denial-of-service, or load testing;
- use social engineering.

This work was therefore a passive security review and responsible-disclosure assessment, not a full penetration test.

3. Skills Demonstrated

This case study demonstrates practical work in:

- API attack-surface review;
- multi-tenant authorization analysis;
- BOLA/IDOR threat modeling;
- HTTP semantics and unsafe method usage;
- CSRF reasoning for cookie-based sessions;
- security review of REST and JSON-RPC interfaces;
- PII and data-minimization analysis;
- secret-handling and logging hygiene;
- risk classification with uncertainty clearly separated from confirmed facts;
- remediation planning and safe validation design;
- responsible disclosure and scope control.

4. Methodology

The assessment used publicly available materials only. The review process included:

1. identifying public application and API documentation;
2. reviewing authentication mechanisms and documented roles;
3. mapping endpoints that accept project IDs, analytics IDs, and other object identifiers;
4. reviewing request and response schemas for sensitive fields;
5. comparing HTTP methods with the semantics of the operations performed;
6. identifying token-like values and debug-oriented fields in public materials;
7. separating directly documented behavior from behavior that could only be verified at runtime;
8. creating a safe internal validation plan based on two isolated test tenants.

A complete review of every JavaScript bundle, source map, manifest, and runtime configuration was outside the completed scope. Therefore, no claim was made that the entire frontend was free of secrets or internal endpoints.

5. Finding Status Model

The report uses the following statuses:

- **Confirmed in public documentation** - behavior is directly visible in current public documentation.
- **Requires runtime validation** - the risk is technically plausible, but the existence of a vulnerability cannot be established without a controlled authorized test.
- **Unverified / historical specification** - the item appeared in a previously reviewed API description but was not present in the current public documentation at the time of final review.
- **Informational** - the observation is not necessarily a vulnerability by itself, but affects security posture or the impact of other weaknesses.

6. Findings Summary

ID	Observation	Status	Potential Impact
F-01	Object-level authorization for an audience-intersection analytics endpoint	Requires runtime validation	High if cross-tenant access is possible
F-02	Historical DELETE operation affecting users of an analytics project	Unverified / historical specification	High/Critical only if route exists and lacks ownership checks

ID	Observation	Status	Potential Impact
F-03	Operational fields exposed by a social-group listing API	Confirmed in public documentation	Informational/Low
F-04	State-changing operations implemented with HTTP GET	Confirmed in public documentation	Medium; higher for cookie sessions without effective CSRF controls
F-05	Historical public project-query endpoint	Unverified / historical specification	Medium if it exposes another tenant's configuration
F-06	JSON-RPC token handling and possible internal search/debug data	Partially confirmed / requires verification	Low/Medium
F-07	Token-like examples in public documentation	Requires verification	Informational/Low unless any value is real
F-08	HTTP 200 used for some application errors	Confirmed for some errors	Low
F-09	PII fields present in documented API schemas	Confirmed in public documentation	Impact amplifier for authorization failures
F-10	Frontend security review was not exhaustive	Informational	Informational

7. Detailed Findings

F-01. Object-Level Authorization for Audience Intersection

Generalized endpoint: GET /api/analytics/audience/intersection

The endpoint accepts identifiers for analytics projects. Public documentation confirms the endpoint and the use of object identifiers, but documentation alone does not establish whether every supplied identifier is checked against the current tenant.

Security hypothesis: If the backend validates only the API token but does not verify ownership of each referenced analytics project, a client could potentially obtain analytics derived from another tenant's object. That would represent a Broken Object Level Authorization (BOLA/IDOR) issue.

Status: Requires runtime validation.

Potential severity: High only if a controlled test proves cross-tenant access.

Safe validation approach:

- create isolated test tenants A and B;
- create an analytics project under tenant A;
- call the endpoint with tenant B's token and tenant A's object ID;
- expect 403, 404, or an equivalent denial without any data leakage;
- verify the corresponding authorization decision in audit logs.

Recommended remediation if confirmed: Perform object-level authorization for every supplied object ID, not only token validation.

F-02. Historical DELETE Operation Affecting Analytics-Project Users

A previously reviewed API description contained a route similar to:

```
DELETE /api/analytics-project/{id}/users
```

The historical description suggested that the operation could be particularly sensitive because it affected project data. However, the route was not present in the current public documentation during the final review.

Status: Unverified / historical specification.

The correct security questions are:

1. Does the route still exist in the current backend?
2. If it exists, is {id} checked against the current tenant?
3. Can the action affect data outside the caller's tenant?
4. Is the operation fully audited?
5. Is recovery possible if the deletion is accidental or unauthorized?

Potential severity: High or Critical only if the route exists and controlled testing confirms cross-tenant modification or deletion.

F-03. Operational Fields Exposed by an External Listing API

Generalized endpoint: GET /api/social-groups

The documented response contained fields that appear operational in nature, such as access state, crawler capability, error counters, internal priority, and regulatory/status flags.

The presence of these fields is confirmed by documentation, but their security impact depends on whether the resource is a global service directory or tenant-specific data.

Status: Confirmed in public documentation.

Potential severity: Informational / Low.

Recommendation: Maintain separate external and internal/operator DTOs. Expose only fields required by external clients.

F-04. State-Changing Operations Implemented with HTTP GET

Generalized endpoints:

```
GET /api/social-group/{id}/subscribe
GET /api/social-group/{id}/unsubscribe
```

These operations change application state but are exposed as HTTP GET requests.

This is a confirmed architectural observation. GET is expected to be safe from the perspective of HTTP semantics, and state-changing GET routes can create problems with prefetching, link previews, security scanners, caches, retries, and browser behavior.

There is also a potential CSRF concern when such an operation is authenticated using cookies that the browser sends automatically. If the endpoint requires an explicit bearer token in the Authorization header, classic CSRF exposure is significantly lower.

Status: Confirmed in public documentation.

Potential severity: Medium; potentially higher if a privileged cookie-based session can execute the action without effective CSRF protection.

Recommendation: Use POST, PUT, or DELETE for state-changing operations. For browser sessions, apply CSRF protection and appropriate Origin/Referer validation where relevant.

F-05. Historical Public Project-Query Endpoint

A previously reviewed specification contained a route resembling:

```
GET /api/search-config?project={id}
```

The historical description suggested that it could return a project's internal search configuration. This route was not confirmed in the current public documentation during the final review.

Status: Unverified / historical specification.

If the route exists, the internal team should verify:

- whether authentication is required;
- whether object-level authorization is performed;
- whether keywords, exclusion rules, filters, or internal search-field names are exposed;
- whether project identifiers can be enumerated;
- whether rate limiting and monitoring are applied.

Potential severity: Medium if the endpoint is public and exposes configuration belonging to another tenant.

F-06. JSON-RPC Token Handling and Internal Debug Data

Public documentation confirmed a JSON-RPC variant for part of the API. A previously reviewed specification also indicated that a token might be supplied inside JSON-RPC parameters and that a serialized internal search query could potentially be returned in a debug-oriented field.

Those implementation details required re-validation against the current API specification and runtime behavior.

Status: Partially confirmed / requires verification.

If tokens are accepted in request bodies, the review should verify that secrets are consistently redacted from application logs, APM traces, error contexts, reverse-proxy logs, and debugging tools.

If internal search/query structures are returned to external clients, they may reveal internal field names, filters, index assumptions, or other implementation details useful for reconnaissance.

Recommendation: Prefer Authorization: Bearer <token> where possible, apply secret redaction consistently, and avoid exposing internal query structures to external clients unless explicitly required.

F-07. Token-Like Examples in Public Documentation

Public examples contained strings that visually resembled access tokens, refresh tokens, webhook secrets, or other credentials.

The values were intentionally not tested against third-party services because doing so would have exceeded the passive scope.

Status: Requires verification.

Potential severity: Informational / Low if all values are placeholders; materially higher if any value is an active credential.

Recommendation: Use obvious non-functional placeholders, scan documentation and example fixtures for secrets, and rotate any credential discovered to be real.

F-08. HTTP 200 Used for Some Application Errors

Documentation showed a legacy pattern in which some application or business errors were returned with HTTP 200 while the actual error condition was represented inside the JSON response body.

This confirms the pattern for some application errors, but does not prove that authorization failures use the same behavior.

Status: Confirmed for some application errors; authorization behavior requires verification.

Security impact:

- client SDKs may misclassify failures as successful requests;
- metrics based only on HTTP status codes lose useful security signal;
- SIEM, WAF, and alerting rules may require body parsing;
- authorization and abuse detection become harder to standardize.

Recommendation: Use appropriate HTTP status codes for new APIs. Where legacy contracts cannot be changed, explicitly log and classify success: false, authorization failures, and other security-relevant outcomes.

F-09. PII Fields Increase the Impact of Authorization Failures

Public API schemas contained fields that may identify users, including names, location, phone, email, date of birth, profile links, and audience-related information.

The presence of PII is not a vulnerability by itself. However, it significantly increases the impact of any object-level authorization failure or bulk-data exposure.

Status: Confirmed in public documentation; contextual risk amplifier.

Recommendation:

- minimize external DTO fields;
- avoid logging phone numbers, email addresses, and other PII unless required;
- apply rate limiting to bulk-reading endpoints;
- maintain audit logs for sensitive exports and queries;
- verify ownership on every endpoint returning tenant-specific user data.

F-10. Frontend Security Review Was Not Exhaustive

Public HTML pages were reviewed for obvious token-like values, but the assessment did not complete a full review of every JavaScript bundle, source map, manifest, or runtime configuration.

The correct conclusion is therefore limited: no confirmed private secrets were identified in the reviewed public HTML, but the absence of secrets across the complete frontend asset set was not established.

Status: Informational.

8. Recommended Internal Triage Plan

Priority 1 - Object-Level Authorization

Using isolated test tenants, review every endpoint that accepts tenant-owned object IDs. The core question for each request is:

Does the backend verify that the authenticated subject is authorized to access this specific object?

Priority 2 - State-Changing GET Operations

Replace unsafe GET mutations where possible. For cookie-based privileged sessions, review CSRF tokens, SameSite/Secure/HttpOnly flags, Origin/Referer validation, confirmation flows, and audit logs.

Priority 3 - External DTOs and Data Minimization

Review externally exposed operational metadata and PII. Separate public/client DTOs from operator/admin DTOs.

Priority 4 - Error Handling and Observability

Review HTTP 401/403/404/429 behavior, authorization-failure metrics, rate limiting, anti-enumeration controls, correlation IDs, and security-event logging.

Priority 5 - Secrets and Frontend Assets

Under explicit authorization, review JavaScript bundles, source maps, runtime JSON configuration, manifests, documentation examples, OpenAPI examples, and token redaction in logs/APM.

9. Safe Plan for an Authorized Follow-Up Pentest

If the organization wanted to validate the hypotheses, the safest follow-up would be a tightly scoped authorized test using:

1. two dedicated test tenants;
2. separate test API tokens;
3. synthetic project and analytics data;
4. no production PII;
5. explicit permission for negative authorization tests between tenant A and tenant B;
6. a defined list of allowed endpoints;
7. no destructive testing outside synthetic data;
8. enabled audit logging;
9. an emergency contact for unexpected impact.

This would allow the team to verify BOLA/IDOR hypotheses without exposing real customer data or causing production risk.

10. Security Architecture Recommendations

Centralized Object-Level Authorization

Avoid relying on individual controllers to implement ownership checks independently. A robust flow is:

```
authenticate subject -> resolve object -> verify tenant/ownership -> verify scope ->
log decision -> execute operation
```

Deny by Default

Any endpoint operating on a tenant-owned object ID should be private by default unless explicitly designed otherwise.

Separate External and Internal DTOs

Do not expose internal operator/admin fields through external client DTOs unless they are required for a documented business use case.

Minimize PII

Return only the data required for the specific business operation.

Maintain Security-Focused Audit Logs

For sensitive actions, record the subject, tenant, non-secret token identifier, object ID, endpoint, authorization decision, number of affected records, source IP, user agent, and correlation ID.

Rate Limiting and Anti-Enumeration

Monitor for repeated 403/404 responses, sequential object-ID access, repeated attempts to access objects owned by other tenants, and unusually large export or bulk-read activity.

Secret Hygiene

Apply secret scanning and redaction to documentation, OpenAPI files, examples and fixtures, application logs, APM, error tracking, and CI/CD workflows.

11. Outcome and Lessons

The strongest conclusions supported by public documentation were:

- state-changing operations were exposed through GET;
- some external responses exposed operational metadata;
- a legacy HTTP 200 error-handling pattern existed for some application errors;

- documented schemas included PII fields that would increase the impact of an authorization failure.

The most important unresolved question was object-level authorization across tenant-owned resources. Public documentation was not sufficient to claim that ownership checks were missing, so the report treated this as a hypothesis requiring an authorized runtime test.

Several historical routes and implementation details were intentionally downgraded to unverified status when they could not be confirmed in the current public documentation.

The key lesson from this engagement was that a useful security review should be precise about evidence. Separating confirmed observations from runtime hypotheses makes the report more credible, reduces false claims, and gives the engineering team a practical validation path.

12. References

- OWASP API Security Top 10 - Broken Object Level Authorization (BOLA)
- RFC 9110 - HTTP Semantics and safe methods
- OWASP Cross-Site Request Forgery Prevention Cheat Sheet
- RFC 6750 - The OAuth 2.0 Authorization Framework: Bearer Token Usage